Training Deep Networks

Machine Learning course notes Sources: listed under References

What this file is really about:

$$\text{a noisy gradient} \rightarrow \text{momentum} \rightarrow \text{per-coordinate scaling} \rightarrow \text{schedules} \rightarrow \text{regularise what you c}$$

How to use this file. Part A builds each optimiser from plain gradient descent. Part B shows the code. Part C is practice with answers.

Step-up path

You need. Gradient descent and neural networks.

1. See why plain descent struggles in valleys.
2. Average gradients with momentum.
3. Scale each coordinate with RMSProp and Adam.
4. Schedule the learning rate.
5. Diagnose first, and then regularise.

You can stop when you can read a loss curve and say what to change next.

Contents

Part A Maths From the Ground Up

A1 Why plain gradient descent struggles

On a long narrow valley the gradient points mostly across the valley, not along it. A step size small enough not to bounce across the walls is then too small to make progress along the floor. Minibatches add noise on top of this.

A2 An exponentially weighted average

Averaging recent values smooths noise:

$$v_t = \beta v_{t-1} + (1-\beta)x_t.$$

The weight on a value k steps old is $(1-\beta)\beta^k$, and the memory is about $1/(1-\beta)$ steps. $\beta = 0.9$ remembers about 10 steps.

> **Careful.** Starting from $v_0 = 0$ biases early averages low. Dividing by $1-\beta^t$ corrects this.

A3 Momentum

Average the gradients rather than the positions:

$$v_t = \beta v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t.$$

Across a valley the gradient keeps changing sign, so it cancels. Along the floor it keeps the same sign, so it builds up.

A4 RMSProp

Give each coordinate its own scale, using a running average of squared gradients:

$$s_t = \rho s_{t-1} + (1-\rho)g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t}+\epsilon} g_t.$$

A coordinate with large gradients takes smaller steps, and one with small gradients takes larger steps.

A5 Adam

Adam combines both, with a correction for the zero start:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad s_t = \beta_2 s_{t-1} + (1-\beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon}.$$

Common defaults are $\beta_1 = 0.9$ and $\beta_2 = 0.999$.



Plain descent zigzags and crawls. Momentum and Adam reach the minimum sooner on the same valley.

A6 Learning-rate schedules

- **Step decay** drops the rate at set epochs.
- **Cosine decay** follows $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t / T))$.
- **Warmup** raises the rate linearly over the first steps, which protects the early, unstable phase.

A7 Diagnose before you regularise

Fit on training data, make choices on validation data, and open the test set once. If training error is high, the model underfits, and regularising makes that worse. If training error is low and validation error is high, it overfits, and regularisation helps.

A8 Regularisers

- **Weight decay** adds $\frac{\lambda}{2}\|\theta\|^2$ to the loss, so each step shrinks the weights by a factor $(1 - \eta\lambda)$.
- **Early stopping** keeps the weights from the best validation epoch.
- **Dropout** zeroes each unit with probability p during training. Dividing the kept activations by $1 - p$ preserves the expected value. At test time all units are used.
- **Data augmentation** adds transformed copies that keep the label.
- **Label smoothing** replaces a target of 1 with $1 - \varepsilon + \varepsilon/K$ and each other target with ε/K.
- **Mixup** trains on $\tilde{x} = \lambda x_i + (1 - \lambda) x_j$ with the labels mixed in the same proportion.

A9 Why momentum helps: a quadratic analysis

On a quadratic with curvatures between μ and L, gradient descent converges at rate $(\kappa - 1)/(\kappa + 1)$ with the best step $2/(L + \mu)$, where $\kappa = L/\mu$. Momentum, with best settings, improves this to

$$\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}.$$

With $\kappa = 100$, the rates are 0.98 and 0.82. Reaching a factor of 1000 takes about 340 steps against about 35.

A10 Adam's bias correction, derived

Adam starts its average at $m_0 = 0$. After t steps,

$$m_t = (1 - \beta) \sum_{i=1}^{t} \beta^{t-i} g_i.$$

If the gradients have constant mean $\bar{g}$, the expected value is $\bar{g}(1-\beta)\sum_{k=0}^{t-1}\beta^k = \bar{g}(1-\beta^t)$. So m_t is too small by exactly that factor, and dividing by $1-\beta^t$ corrects it. This matters most in early steps.

A11 Weight decay is not always L2

For plain SGD, adding $\frac{\lambda}{2}\|w\|^2$ to the loss is identical to shrinking weights each step. For Adam it is not: the penalty's gradient gets divided by the adaptive scale, so large-gradient weights are barely regularised. **AdamW** subtracts $\eta\lambda w$ directly, after the adaptive step, which keeps the decay uniform.

A12 Dropout and its scale

With keep probability q, an output is $z \cdot b$ where $b \sim \text{Bernoulli}(q)$, so $\mathbb{E}[zb] = qz$. Dividing by q during training makes the expected output equal to z, so nothing needs changing at test time. The variance added is $z^2(1-q)/q$, which is the source of the regularisation.

A13 A complete diagnosis procedure

1. Check that the initial loss equals its theoretical value.
2. Overfit one batch.
3. Find a learning rate with a range test.
4. Train, and plot training and validation loss.
5. Training loss high: raise capacity or the learning rate, and check the data.
6. Gap large: add data, augmentation, regularisation.
7. Only then tune finer settings.

Part B In Code

B1 Choosing an optimiser

```
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
```

B2 Early stopping

```
if val_loss < best:
    best, wait = val_loss, 0
    torch.save(model.state_dict(), 'best.pt')
else:
    wait += 1
    if wait >= patience:
        break
```

B3 Optimisers from scratch

Complete implementations, compared on an ill-conditioned quadratic.

```python
import numpy as np

A = np.diag([1.0, 100.0])
grad = lambda w: A @ w
loss = lambda w: 0.5 * w @ A @ w


def sgd(lr, steps=200):
    w = np.array([1.0, 1.0])
    for _ in range(steps):
        w = w - lr * grad(w)
    return loss(w)


def momentum(lr, b=0.9, steps=200):
    w = np.array([1.0, 1.0]); v = 0 * w
    for _ in range(steps):
        v = b * v + grad(w)
        w = w - lr * v
    return loss(w)


def rmsprop(lr, r=0.9, steps=200):
    w = np.array([1.0, 1.0]); s = 0 * w
    for _ in range(steps):
        g = grad(w)
        s = r * s + (1 - r) * g ** 2
        w = w - lr * g / np.sqrt(s + 1e-8)
    return loss(w)


def adam(lr, b1=0.9, b2=0.999, steps=200):
    w = np.array([1.0, 1.0]); m = 0 * w; s = 0 * w
    for t in range(1, steps + 1):
        g = grad(w)
        m = b1 * m + (1 - b1) * g
```

```
        s = b2 * s + (1 - b2) * g ** 2
        mh, sh = m / (1 - b1 ** t), s / (1 - b2 ** t)
        w = w - lr * mh / (np.sqrt(sh) + 1e-8)
    return loss(w)


print('sgd     ', sgd(0.0198))
print('momentum', momentum(0.012))
print('rmsprop ', rmsprop(0.01))
print('adam    ', adam(0.05))
assert momentum(0.012) < sgd(0.0198)
```

B4 A learning-rate range test

```
import torch
from torch import nn

torch.manual_seed(0)
X = torch.randn(512, 10); y = (X[:, 0] + X[:, 1] > 0).long()
net = nn.Sequential(nn.Linear(10, 64), nn.ReLU(), nn.Linear(64, 2))
opt = torch.optim.SGD(net.parameters(), lr=1e-4)
history = []
for lr in torch.logspace(-4, 1, 100).tolist():
    for g in opt.param_groups:
        g['lr'] = lr
    loss = nn.functional.cross_entropy(net(X), y)
    opt.zero_grad(); loss.backward(); opt.step()
    history.append((lr, loss.item()))
    if loss.item() > 4 * history[0][1] or loss != loss:
        break

best = min(history, key=lambda h: h[1])
print('lowest loss at lr', round(best[0], 4), '- use about a tenth of it')
```

Part C Practice

P1. $\beta = 0.9$. About how many steps does the average remember?

$1/(1 - 0.9) = 10$.

P2. $v = 2$, $g = 3$, $\beta = 0.9$. Find the new momentum, in the form $v \leftarrow \beta v + g$.

$1.8 + 3 = 4.8$.

P3. $\eta = 0.01$ and $\lambda = 0.1$. By what factor does weight decay shrink a weight each step?

$1 - 0.001 = 0.999$.

P4. Dropout with $p = 0.5$. By what factor are kept activations scaled in training?

$1/(1 - 0.5) = 2$.

P5. Label smoothing with $\varepsilon = 0.1$ and 10 classes. Find the target for the true class.

$0.9 + 0.01 = 0.91$.

P6. Training loss is high and validation loss is about the same. Should you add dropout?

No. The model is underfitting, so it needs more capacity or more training.

P7. With $\beta = 0.9$, what is the bias factor of m_t at $t = 2$?

$1 - 0.9^2 = 0.19$, so raw m_2 is about a fifth of its true size.

P8. Why does AdamW differ from Adam with an L2 penalty?

Adam divides the L2 gradient by its adaptive scale. AdamW applies the decay directly, so it is the same for every weight.